Practical 2

Aim: Develop an Android application demonstrating the use of Programming Resources (colors.xml, themes.xml, strings.xml, dimens.xml, drawable resources) and apply them in UI components.

Android resources store reusable values and assets separately from application code. colors.xml defines colours, strings.xml stores text, dimens.xml stores dimensions, themes.xml defines the application theme, and drawable resources store images or drawable shapes. Using resources makes an application easier to maintain and customise.

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:text="Hello Harsh!"
        android:textColor="#0B3BB3"
        android:textSize="34sp" />

    <ImageButton
        android:id="@+id/imageButton"
        android:layout_width="100dp"
        android:layout_height="80dp"
        app:srcCompat="@android:drawable/btn_star_big_on" />

    <ImageView
        android:id="@+id/imageView"
        android:layout_width="match_parent"
        android:layout_height="448dp"
        app:srcCompat="@drawable/img" />

</LinearLayout>
```

MainActvity.kt

```
package com.example.pract1

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="colorPrimary">#008577</color>
    <color name="colorPrimaryDark">#00574B</color>
```

```
    <color name="colorAccent">#D81B60</color>

</resources>
```

Theme.xml

```
<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.Pract1" parent="Theme.AppCompat.Light.DarkActionBar">
        <!-- Customize your light theme here. -->
        <!-- <item name="colorPrimary">@color/my_light_primary</item> -->
        <item name="colorPrimary">@color/colorPrimary</item>
        <item name="colorPrimaryDark">@color/colorPrimaryDark</item>
        <item name="colorAccent">@color/colorAccent</item>

    </style>

    <style name="Theme.Pract1" parent="Base.Theme.Pract1" />
</resources>
```

String.xml

```
<resources>
    <string name="app_name">Practical2</string>
    <string name="full_name">Harsh Sakpal </string>
    <string name = "roll_no">18</string>
</resources>
```

Output:



Practical 3

Aim: Develop an Android application demonstrating complete Activity Lifecycle methods and analyze execution using Logcat.

An Activity passes through lifecycle methods as it is created, displayed, paused, stopped, restarted, or destroyed. Logcat displays application logs and helps developers observe lifecycle execution and debug errors.

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello Harsh!"
        android:textSize="20sp"
        android:textStyle="bold"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

MainActivity.kt

```
package com.example.pract3

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.util.Log

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        print("Oncreate")
    }

    override fun onStart() {
        super.onStart()
        print("onStart")
    }

    override fun onResume() {
        super.onResume()
        print("onResume")
    }

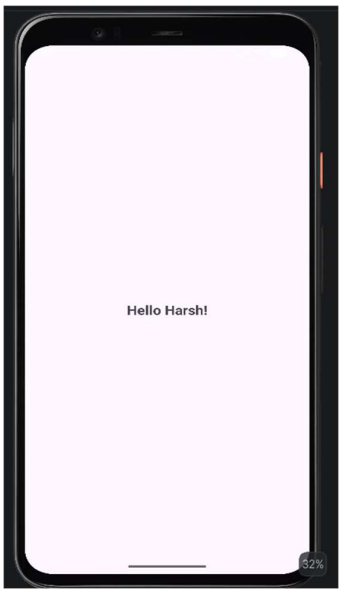
    override fun onPause() {
        super.onPause()
        print("onPause")
    }

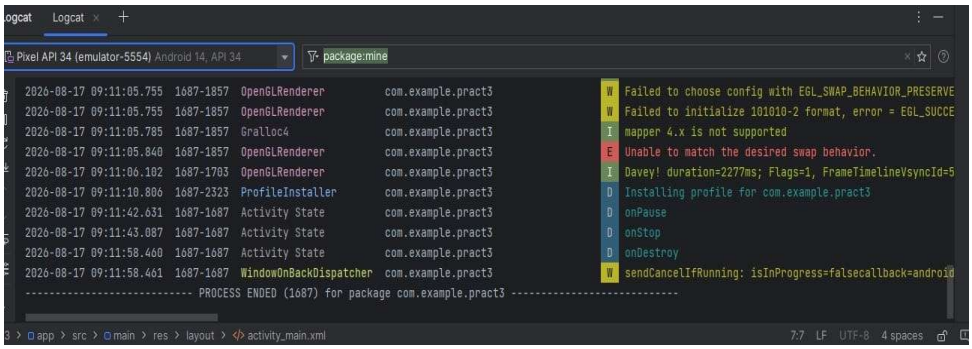
    override fun onRestart() {
        super.onRestart()
        print("onRestart")
    }

    override fun onStop() {
        super.onStop()
        print("onStop")
    }
```

```
    override fun onDestroy() {
        super.onDestroy()
        print("onDestroy")
    }
fun print(msg:String)
{
    Log.d("Activity State",msg)
}
}
```

Output





Practical 4

Aim: Develop an Android application to create and integrate a Fragment and demonstrate Fragment lifecycle methods.

A Fragment is a reusable portion of an application's user interface that is hosted inside an Activity. It has lifecycle callbacks such as onCreate(), onCreateView(), onStart(), onResume(), onPause(), onStop(), onDestroyView(), and onDestroy(). Fragments help divide a screen into modular and manageable sections.

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/fragmentContainerView"
        android:name="com.example.pract4.FirstFragment"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />
</FrameLayout>
```

MainActivity.kt

```
package com.example.pract4

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

first_fragment.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".FirstFragment">

    <!-- TODO: Update blank fragment layout -->
    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:text="Harsh Sakpal"
        android:textColor="#3F51B5"
        android:textSize="24sp"
        android:textStyle="bold|italic" />

    <TextView
        android:id="@+id/FirstFragment"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:text="First Fragment"
        android:textColor="#FFC107"
        android:textSize="24sp"
        android:textStyle="bold" />

</LinearLayout>
```

FirstFragment.kt

```
package com.example.pract4

import android.os.Bundle
import android.util.Log
import androidx.fragment.app.Fragment
import android.view.LayoutInflater
import android.view.View
```

```kotlin
import android.view.ViewGroup

class FirstFragment : Fragment() {
    // TODO: Rename and change types of parameters
    private val TAG = "FragmentLife"

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d(TAG, "OnCreate")
        }


    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        Log.d(TAG, "OnCreateView")
        return inflater.inflate(R.layout.fragment_first, container, false)
    }

    override fun onStart(){
        super.onStart()
        Log.d(TAG, "onStart")
    }
    override fun onResume(){
        super.onResume()
        Log.d(TAG, "onResume")
    }
    override fun onPause(){
        super.onPause()
        Log.d(TAG, "onPause")
    }
    override fun onStop(){
        super.onStop()
        Log.d(TAG, "onStop")
    }
    override fun onDestroyView(){
        super.onDestroyView()
        Log.d(TAG, "onDestroyView")
    }
    override fun onDestroy(){
        super.onDestroy()
        Log.d(TAG, "onDestroy")
    }
}
```

Output:

```
2026-07-24 15:59:12.997 11660-11660 FragmentLife            com.example.pract4                   D  OnCreate
2026-07-24 15:59:13.041 11660-11660 FragmentLife            com.example.pract4                   D  OnCreateView
2026-07-24 15:59:13.075 11660-11660 FragmentLife            com.example.pract4                   D  onStart
2026-07-24 15:59:13.081 11660-11660 FragmentLife            com.example.pract4                   D  onResume
2026-07-24 16:00:11.262 11660-11660 FragmentLife            com.example.pract4                   D  onPause
2026-07-24 16:00:11.271 11660-11660 FragmentLife            com.example.pract4                   D  onStop
2026-07-24 16:00:11.311 11660-11660 FragmentLife            com.example.pract4                   D  onDestroyView
2026-07-24 16:00:11.328 11660-11660 FragmentLife            com.example.pract4                   D  onDestroy
---------------------------- PROCESS ENDED (11660) for package com.example.pract4 ----------------------------
```

Practical 5

Aim: Develop an Android application implementing Fragment-based UI navigation between multiple screens.

Fragment-based navigation allows an application to move between different screens without creating a separate Activity for every screen. FragmentManager and FragmentTransaction are used to add, replace, or remove fragments. The back stack can preserve navigation history and allow users to return to the previous fragment.

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/fragmentContainerView2"
        android:name="com.example.practical5.FirstFragment"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />
</FrameLayout>
```

MainActivity.kt

```
package com.example.practical5

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

first_fragment.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".FirstFragment">

    <!-- TODO: Update blank fragment layout -->
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="First Screen"
        android:textColor="#081F75"
        android:textSize="24sp" />

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:text="Harsh Sakpal "
        android:textColor="#FF9800"
```

```
        android:textSize="24sp"
        android:textStyle="bold" />

    <Button
        android:id="@+id/btnNext"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Next"
        android:textSize="24sp"
        android:textStyle="bold" />

</LinearLayout>
```

FirstFragment.kt

```
package com.example.practical5

import android.os.Bundle
import androidx.fragment.app.Fragment
import android.view.*
import android.widget.Button

//
class FirstFragment : Fragment() {


    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        // Inflate the layout for this fragment
        val view = inflater.inflate(R.layout.fragment_first, container, false)
        view.findViewById<Button>(R.id.btnNext).setOnClickListener{
            parentFragmentManager.beginTransaction()
                .replace(R.id.fragmentContainerView2, SecondFragment())
                .addToBackStack(null)
                .commit()
        }
        return view
    }


}
```

second_fragment.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".SecondFragment">

    <!-- TODO: Update blank fragment layout -->
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:text="Second Screen"
        android:textColor="#081F75"
        android:textSize="24sp" />

    <TextView
        android:id="@+id/textView2"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:text="Harsh Sakpal"
        android:textColor="#FF9800"
        android:textSize="24sp"
        android:textStyle="bold" />

    <Button
```

```
        android:id="@+id/btnBack"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Back"
        android:textSize="24sp"
        android:textStyle="bold" />

</LinearLayout>
```

SecondFragment.kt

```
package com.example.practical5

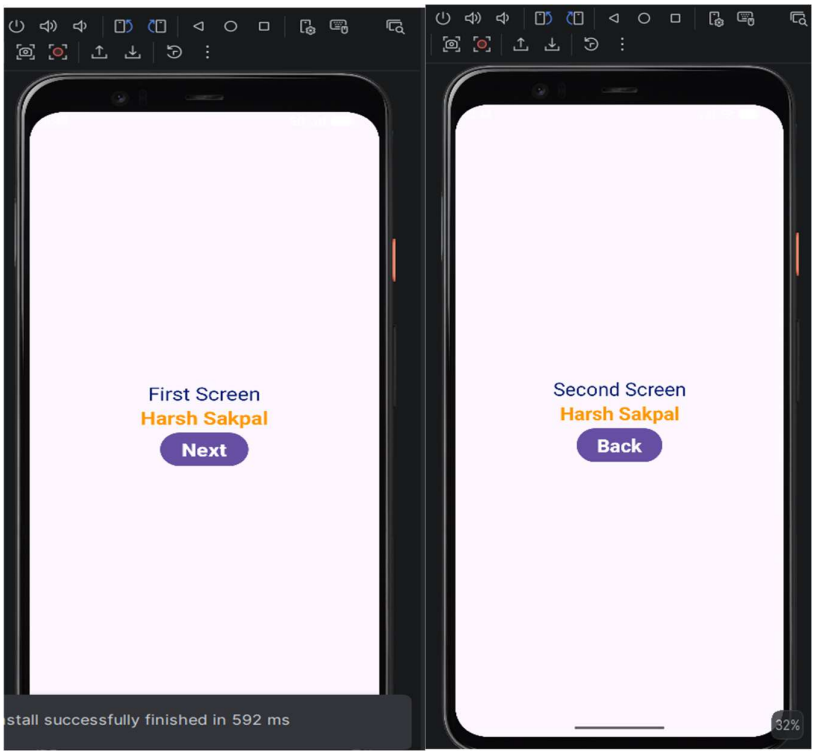
import android.os.Bundle
import androidx.fragment.app.Fragment
import android.view.*
import android.widget.Button


class SecondFragment : Fragment() {


    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        // Inflate the layout for this fragment
        val view = inflater.inflate(R.layout.fragment_second, container, false)
            view.findViewById<Button>(R.id.btnBack).setOnClickListener{
                parentFragmentManager.popBackStack()

            }
        return view
        }
    }
```

Output:



Practical 7

Aim: Develop an Android application using LinearLayout (Horizontal and Vertical) to design structured UI.

LinearLayout arranges its child views in a single row or a single column. The orientation attribute sets the arrangement to horizontal or vertical. It is useful for creating structured forms and screens containing TextViews, EditTexts, and Buttons.

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/textView2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Harsh Sakpalu "
        android:textColor="#3F51B5"
        android:textSize="24sp"
        android:textStyle="bold|italic" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Enter Your Name"
        android:textColor="#FF9800"
        android:textSize="24sp"
        android:textStyle="bold"
        tools:ignore="TextContrastCheck" />

    <EditText
        android:id="@+id/txtName"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:ems="10"
        android:inputType="text"
        tools:ignore="SpeakableTextPresentCheck,TouchTargetSizeCheck" />

    <TextView
        android:id="@+id/textView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="16sp" />

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">

        <Button
            android:id="@+id/btnDisplay"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Display" />

        <Button
            android:id="@+id/btnClear"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
```

```
            android:layout_weight="1"
            android:text="Clear" />
    </LinearLayout>

</LinearLayout>
```

MainActivity.kt

```
package com.example.pract7

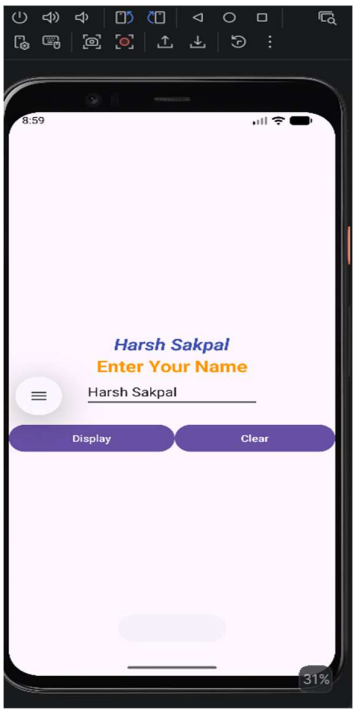
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.Toast
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)
        val etName=findViewById<EditText>(R.id.txtName)
        val btDisplay=findViewById<Button>(R.id.btnDisplay)
        val btClear=findViewById<Button>(R.id.btnClear)
        btDisplay.setOnClickListener {
            val name=etName.text.toString()
            if(name.isNotEmpty()){
                Toast.makeText(this,"Welcome $ ",Toast.LENGTH_SHORT).show()

            }
            else
            {
                Toast.makeText(this,"Kindly enter your Name",Toast.LENGTH_SHORT).show()
            }
        }
        btClear.setOnClickListener {
            etName.text.clear()
        }

    }
}
```

Output:



Practical 8

Aim: Develop an Android application using RelativeLayout to align UI components dynamically.

RelativeLayout positions views in relation to the parent layout or to other views. Attributes such as layout_below, layout_alignParentTop, and layout_toEndOf help control the position of UI components. It can be used to create forms and interfaces with related elements.

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/txtStudent"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Harsh Sakpal"
        android:textColor="#3F51B5"
        android:textSize="24sp"
        android:textStyle="bold|italic" />

    <TextView
        android:id="@+id/txtView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@id/txtStudent"
```

```
        android:text="Enter Your Name"
        android:textColor="#FF9800"
        android:textSize="24sp"
        android:textStyle="bold" />

    <EditText
        android:id="@+id/txtName"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@id/txtView"
        android:ems="10"
        android:inputType="text"
        tools:ignore="SpeakableTextPresentCheck,TouchTargetSizeCheck" />

    <Button
        android:id="@+id/btnDisplay"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@id/txtName"
        android:text="Display" />

    <Button
        android:id="@+id/btnClear"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@id/btnDisplay"
        android:text="Clear" />

</RelativeLayout>
```

MainActivity.kt

```
package com.example.pract8

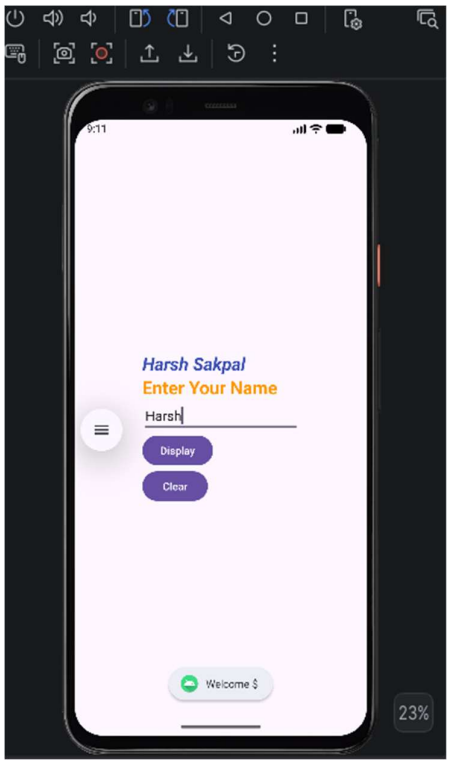
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.Toast
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)
        val etName=findViewById<EditText>(R.id.txtName)
        val btDisplay=findViewById<Button>(R.id.btnDisplay)
        val btClear=findViewById<Button>(R.id.btnClear)
        btDisplay.setOnClickListener {
            val name=etName.text.toString()
            if(name.isNotEmpty()){
                Toast.makeText(this,"Welcome $ ",Toast.LENGTH_SHORT).show()

            }
            else
            {
                Toast.makeText(this,"Kindly enter your Name",Toast.LENGTH_SHORT).show()
            }
        }
        btClear.setOnClickListener {
            etName.text.clear()
        }

    }
}
```

Output:



Practical 9

Aim: Develop an Android application using TableLayout to design a structured Login/Registration Form.

TableLayout organises UI components into rows and columns using TableRow elements. Each row can contain labels, text fields, buttons, or other views. It is useful for designing structured login, registration, and data-entry forms.

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<TableLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    tools:context=".MainActivity">

    <TableRow
        android:layout_width="match_parent"
        android:layout_height="match_parent">

        <TextView
            android:id="@+id/textView"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="User Name"
            android:textSize="24sp"
            android:textStyle="bold" />
```

```
        <EditText
            android:id="@+id/etUserName"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:ems="10"
            android:hint="Enter User Name"
            android:inputType="text"
            android:textSize="20sp"
            tools:ignore="SpeakableTextPresentCheck" />
    </TableRow>

    <TableRow
        android:layout_width="match_parent"
        android:layout_height="match_parent">

        <TextView
            android:id="@+id/textView2"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Password"
            android:textSize="24sp"
            android:textStyle="bold" />

        <EditText
            android:id="@+id/etPassword"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:ems="10"
            android:hint="Enter Password"
            android:inputType="numberPassword"
            android:textSize="20sp"
            tools:ignore="SpeakableTextPresentCheck" />
    </TableRow>
    <TableRow
        android:layout_width="match_parent"
        android:layout_height="match_parent">

        <Button
            android:id="@+id/btnLogin"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Login"
            android:textSize="20sp"
            android:textStyle="bold" />

        <Button
            android:id="@+id/btnCancel"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Cancel"
            android:textSize="20sp"
            android:textStyle="bold" />
    </TableRow>
</TableLayout>
```

MainActivity.kt

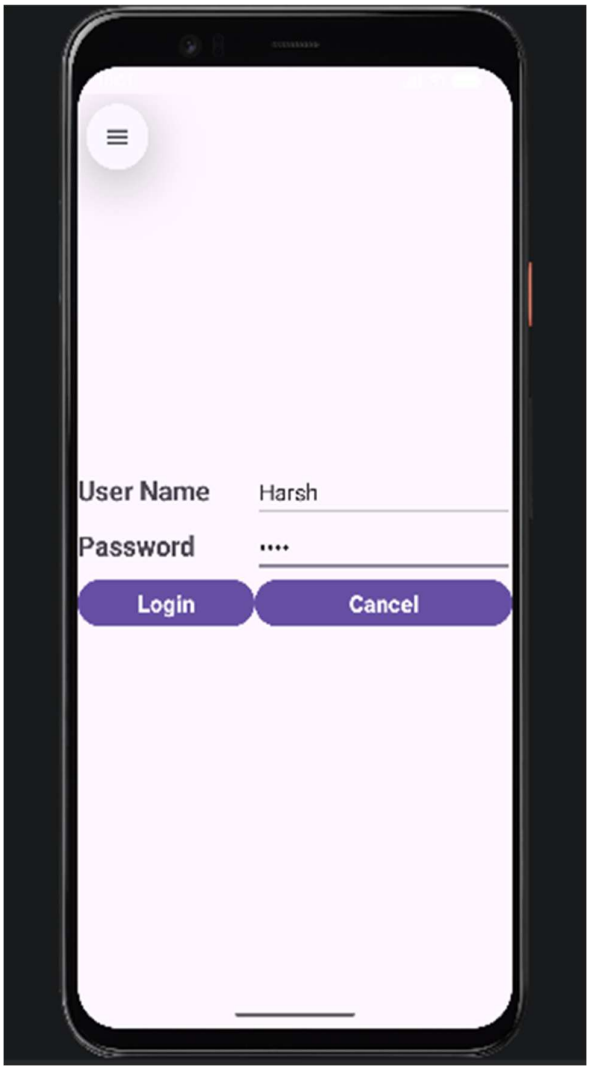
```
package com.example.pract9

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.Toast

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        val etUserName=findViewById<EditText>(R.id.etUserName)
        val etPassword=findViewById<EditText>(R.id.etPassword)
        val btLogin=findViewById<Button>(R.id.btnLogin)
        val btCancel=findViewById<Button>(R.id.btnCancel)
        btLogin.setOnClickListener {
```

```
            val name=etUserName.text.toString()
            val pass=etPassword.text.toString()
            if(name=="Ayesha" && pass=="1720"){
                Toast.makeText(this,"Welcome $name",Toast.LENGTH_SHORT).show()
            }
            else
            {
                Toast.makeText(this,"KIndly Enter Correct Login
details",Toast.LENGTH_SHORT).show()
            }
            btCancel.setOnClickListener {
                etUserName.text.clear()
                etPassword.text.clear()
            }
        }
    }
}
```

Output



Practical 10

Aim: Develop an Android application using ScrollView to display long scrollable content.

ScrollView allows content that is taller than the screen to be viewed by scrolling vertically. It generally contains one direct child layout, which holds the views that need scrolling. It is useful for long forms, instructions, and pages with extensive content.

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity" >

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <ImageView
            android:id="@+id/imageView2"
            android:layout_width="match_parent"
            android:layout_height="775dp"
            app:srcCompat="@drawable/ic_launcher_foreground"
            tools:srcCompat="@drawable/ic_launcher_foreground" />

        <TextView
            android:id="@+id/textView5"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Andriod Programming"
            android:textSize="40sp"
            android:textStyle="bold" />

        <TextView
            android:id="@+id/textView6"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Unit1:Introduction to Andriod"
            android:textSize="40sp"
            android:textStyle="italic" />

        <TextView
            android:id="@+id/textView7"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Unit 2:Actvities and intents"
            android:textSize="40sp" />

        <TextView
            android:id="@+id/textView8"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Unit 3:Layouts"
            android:textSize="40sp" />

        <TextView
            android:id="@+id/textView9"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Unit 4:Fragments"
            android:textSize="40sp" />

        <TextView
```

```
            android:id="@+id/textView10"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Unit 5:SQLite Database"
            android:textSize="40sp" />

        <TextView
            android:id="@+id/textView11"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="unit6: Networking"
            android:textSize="40sp" />

        <TextView
            android:id="@+id/textView12"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Unit 7:Firebase"
            android:textSize="40sp" />

        <TextView
            android:id="@+id/textView13"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Unit 8:Publishing Apps"
            android:textSize="40sp" />

    </LinearLayout>
</ScrollView>
```

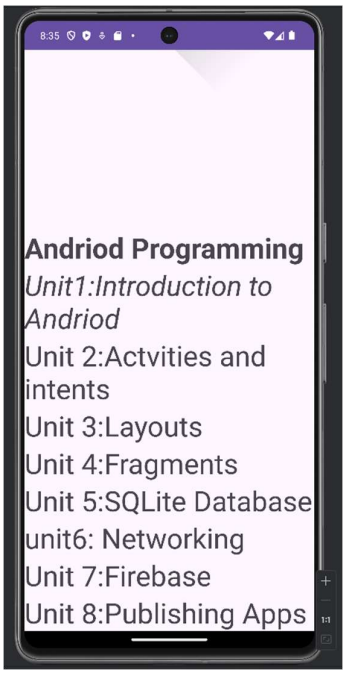
MainActivity.kt

```
package com.example.pract10

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

Output



Practical 13

Aim: Develop an Android application using ListView with ArrayAdapter and implement item click handling.

ListView displays a vertically scrollable list of items. ArrayAdapter connects an array or list of data to the ListView and provides a view for each item. An item-click listener can be used to respond when the user selects an item.

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ListView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/listView"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:entries="@array/list"
    tools:context=".MainActivity" />
```

MainActivity.kt

```
package com.example.pract13

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.ArrayAdapter
import android.widget.ListView
import android.widget.Toast

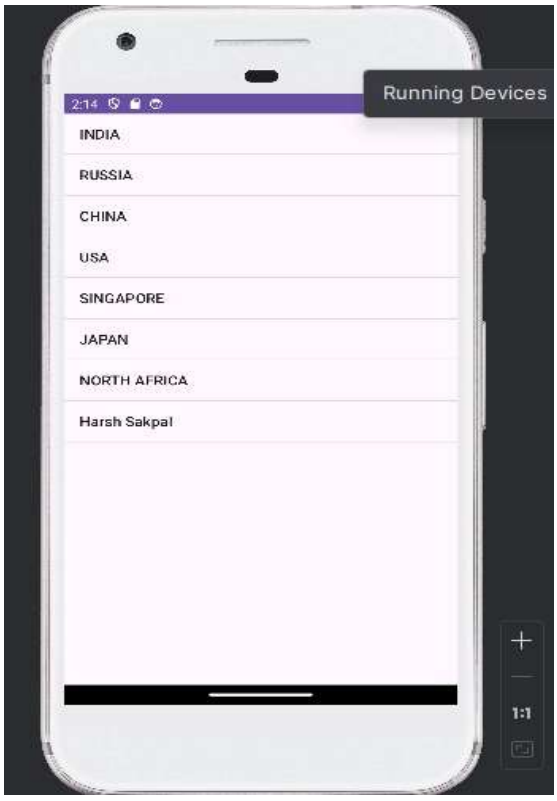
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
```

```
        val listView =findViewById<ListView>(R.id.listView)
        val items=resources.getStringArray(R.array.list)
        val adapter =ArrayAdapter(this,android.R.layout.simple_list_item_1, items)
        listView.adapter=adapter
        listView.setOnItemClickListener { _,_, position, _ ->
            Toast.makeText(this, items[position],Toast.LENGTH_SHORT).show()
        }
    }
}
```

Strings.xml

```
<resources>
    <string name="app_name">pract13</string>
    <array name="list">
        <item>INDIA</item>
        <item>RUSSIA</item>
        <item>CHINA</item>
        <item>USA</item>
        <item>SINGAPORE</item>
        <item>JAPAN</item>
        <item>NORTH AFRICA</item>
        <item>Harsh Sakpal</item>
    </array>
</resources>
```

Output



Practical 14

Aim: Develop an Android application using GridView with Custom Adapter and image selection handling.

GridView displays items in a two-dimensional, scrollable grid of rows and columns. A custom adapter can define how each item, such as an image and a label, is displayed. An item-click listener can handle image or grid-item selection.

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<GridView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/GridView"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:visibility="visible"
    tools:context=".MainActivity">


</GridView>
```

MainActivity.kt

```
package com.example.pract14

import MyAdapter
import android.annotation.SuppressLint
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.GridView
import android.widget.Toast
class MainActivity : AppCompatActivity() {
            private val names = arrayOf(
                "apple",
                "banana",
                "mango",
                "orange"
            )
            private val images =
                intArrayOf(R.drawable.apple, R.drawable.banana, R.drawable.mango, R.drawable.orange)

            @SuppressLint("MissingInflatedId")
            override fun onCreate(savedInstanceState: Bundle?) {
                super.onCreate(savedInstanceState)
                setContentView(R.layout.activity_main)
                val gridView = findViewById<GridView>(R.id.GridView)
                val adapter = MyAdapter(this, names, images)
                gridView.adapter = adapter
                gridView.setOnItemClickListener { _, _, position, _ ->
                    Toast.makeText(this, names[position], Toast.LENGTH_LONG).show()

                }
            }
        }
```

Grid_item.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".MyAdapter">

    <ImageView
```

```
        android:id="@+id/imageView3"
        android:layout_width="80dp"
        android:layout_height="80dp"
        app:srcCompat="@mipmap/ic_launcher" />

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:text="Apple"
        android:textColor="#C51111"
        android:textSize="24sp"
        android:textStyle="bold" />
</LinearLayout>
```

MyAdapter.kt

```
import android.content.Context
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.BaseAdapter
import android.widget.ImageView
import android.widget.TextView
import com.example.pract14.R

class MyAdapter(
    private val context: Context,
    private val names: Array<String>,
    private val images: IntArray
) : BaseAdapter() {

    override fun getCount(): Int = names.size

    override fun getItem(position: Int): Any = names[position]

    override fun getItemId(position: Int): Long = position.toLong()

    override fun getView(position: Int, convertView: View?, parent: ViewGroup?): View {

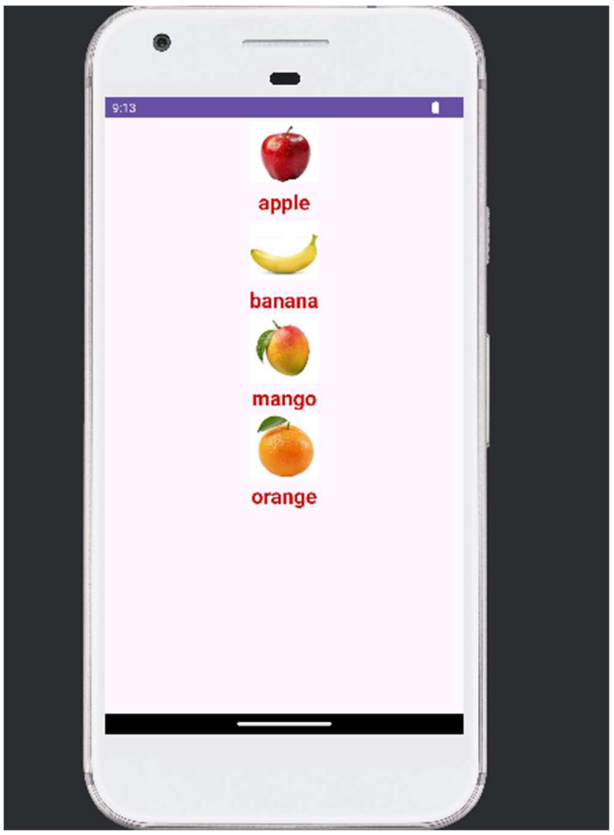
        val view = LayoutInflater.from(context).inflate(R.layout.grid_item, parent, false)

        val image = view.findViewById<ImageView>(R.id.imageView3)
        val text = view.findViewById<TextView>(R.id.textView)

        image.setImageResource(images[position])
        text.text = names[position]

        return view
    }
}
```

Output



MODULE 2

Practical 1

Aim: Develop an Android application implementing AppBar (Toolbar) and Options Menu.

The Toolbar provides a consistent top area for an application's title, navigation controls, and actions. An options menu presents additional actions that users can select. Menu resources and menu callbacks are used to define and handle these actions.

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <androidx.appcompat.widget.Toolbar
        android:id="@+id/toolbar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="?attr/colorPrimary"
        android:minHeight="?attr/actionBarSize"
        android:theme="?attr/actionBarTheme"
        app:title="Harsh Sakpal" />

    <TextView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:gravity="center"
        android:text="Welcome"
        android:textSize="34sp"
        android:textStyle="bold" />

</LinearLayout>
```

Menu.xml

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">

    <item
        android:id="@+id/help"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:title="Help" />
    <item
        android:id="@+id/view"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:title="View">
        <menu>
            <item
                android:id="@+id/logout"
                android:title="Logout" />
        </menu>
    </item>

</menu>
```

MainActivity.kt

```
package com.example.m2pract1

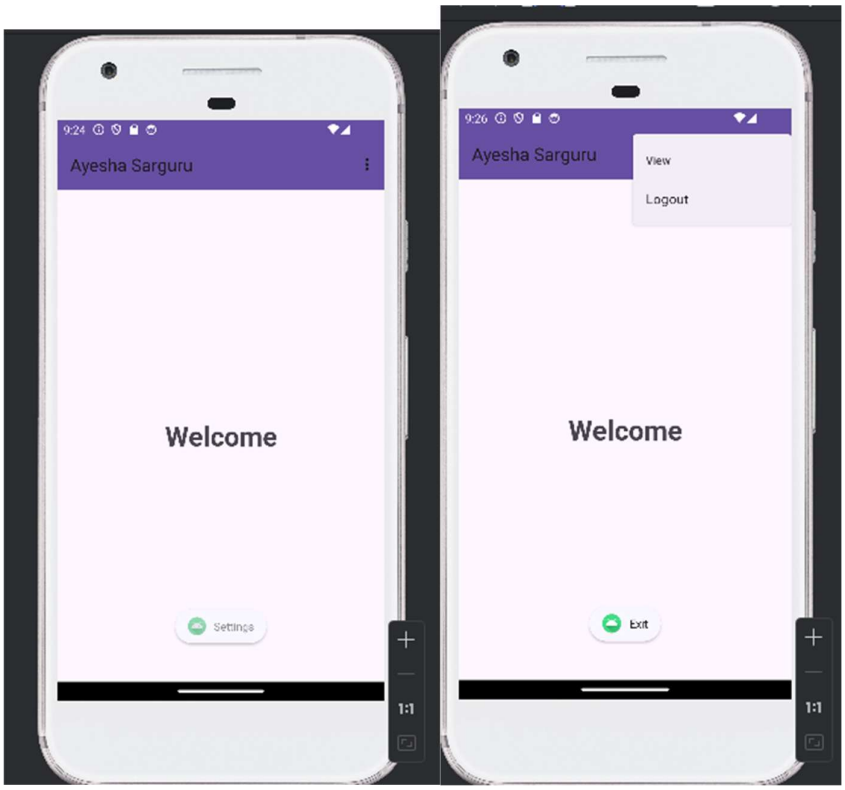
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.view.*
import android.view.Menu
import android.view.MenuItem
import android.widget.Toast
import androidx.appcompat.widget.Toolbar

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val toolbar = findViewById<Toolbar>(R.id.toolbar)
        setSupportActionBar(toolbar)
    }
        override fun onCreateOptionsMenu(menu: Menu): Boolean{
            menuInflater.inflate(R.menu.menu,menu)
            return true
        }

        override fun onOptionsItemSelected(item: MenuItem): Boolean {
            when (item.itemId){
                R.id.help ->{
                    Toast.makeText(this,"Settings",Toast.LENGTH_SHORT).show()
                    return true
                }
                R.id.view ->{
                    Toast.makeText(this, "Exit", Toast.LENGTH_SHORT).show()
                    return true
                }
                R.id.logout ->{
                    Toast.makeText(this, "Logged Out", Toast.LENGTH_SHORT).show()
                    return true
                }
            }
            return super.onOptionsItemSelected(item)
        }
    }
```

Output



Practical 2

Aim: Develop an Android application demonstrating AlertDialog and custom DialogFragment.

An AlertDialog displays a small dialog containing a title, message, and action buttons, such as OK or Cancel. A custom DialogFragment can display a dialog with a customised layout. Dialogs are useful for confirmations, warnings, and short user interactions.

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <Button
        android:id="@+id/btnAlert"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Alert Dialog" />

    <Button
        android:id="@+id/btnCustom"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
```

```
        android:text="Custom Dialog" />

</LinearLayout>
```

Dialog_custom.xml

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="20dp"
    tools:context=".CustomDialogFragment">

    <EditText
        android:id="@+id/etName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:ems="10"
        android:hint="Enter Name"
        android:inputType="text"
        android:textSize="24sp"
        tools:ignore="TouchTargetSizeCheck" />

</LinearLayout>
```

CustomDialogFragment.kt

```
package com.example.m2pract2

import android.app.Dialog
import android.os.Bundle
import android.view.LayoutInflater
import androidx.appcompat.app.AlertDialog
import androidx.fragment.app.DialogFragment

class CustomDialogFragment : DialogFragment() {

    override fun onCreateDialog(savedInstanceState: Bundle?): Dialog {

        val view = LayoutInflater.from(requireContext())
            .inflate(R.layout.dialog_custom, null)

        return AlertDialog.Builder(requireContext())
            .setTitle("Custom Dialog")
            .setView(view)
            .setPositiveButton("OK", null)
            .show()
    }
}
```

MainActivity.kt

```
package com.example.m2pract2

import android.os.Bundle
import android.widget.Button
import androidx.appcompat.app.AlertDialog
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

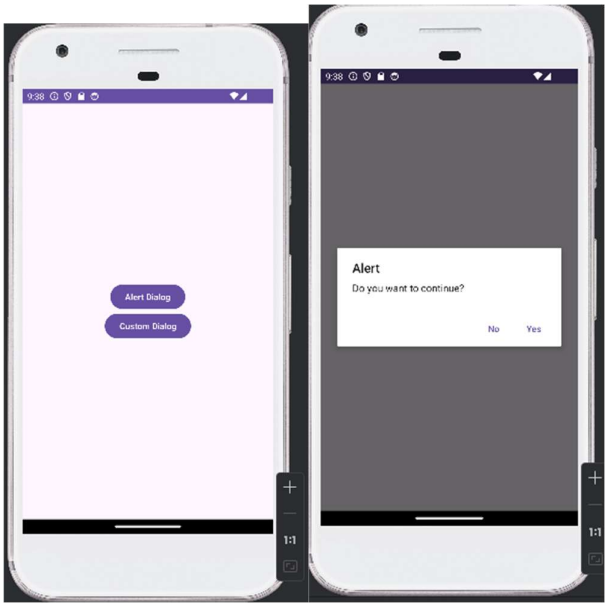
        setContentView(R.layout.activity_main)

        val btnAlert = findViewById<Button>(R.id.btnAlert)
        val btnCustom = findViewById<Button>(R.id.btnCustom)
```

```
        btnAlert.setOnClickListener {
            AlertDialog.Builder(this)
                .setTitle("Alert")
                .setMessage("Do you want to continue?")
                .setPositiveButton("Yes", null)
                .setNegativeButton("No", null)
                .show()
        }

        btnCustom.setOnClickListener {
            CustomDialogFragment().show(supportFragmentManager, "CustomDialog")
        }
    }
}
```

Output:



Practical 4

Aim: Develop an Android application implementing Explicit and Implicit Intents.

An Intent is a messaging object used to request an action from another Android component. An explicit Intent names the component to open, such as a particular Activity. An implicit Intent describes an action, such as opening a webpage or sharing text, and Android finds a suitable application to handle it.

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".MainActivity">
```

```
    <Button
        android:id="@+id/btnExplicit"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Explicit Intent"
        android:textSize="24sp"
        android:textStyle="bold" />

    <Button
        android:id="@+id/btnImplicit"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Implicit Intent"
        android:textSize="24sp"
        android:textStyle="bold" />
</LinearLayout>
```

Activity_second.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".SecondActivity">

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Second Activity"
        android:textColor="#FF9800"
        android:textSize="34sp"
        android:textStyle="bold" />
</LinearLayout>
```

Create Second Activity

MainActivity.kt

```
package com.example.m2pract4

import android.content.Intent
import android.net.Uri
import android.os.Bundle
import android.widget.Button
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

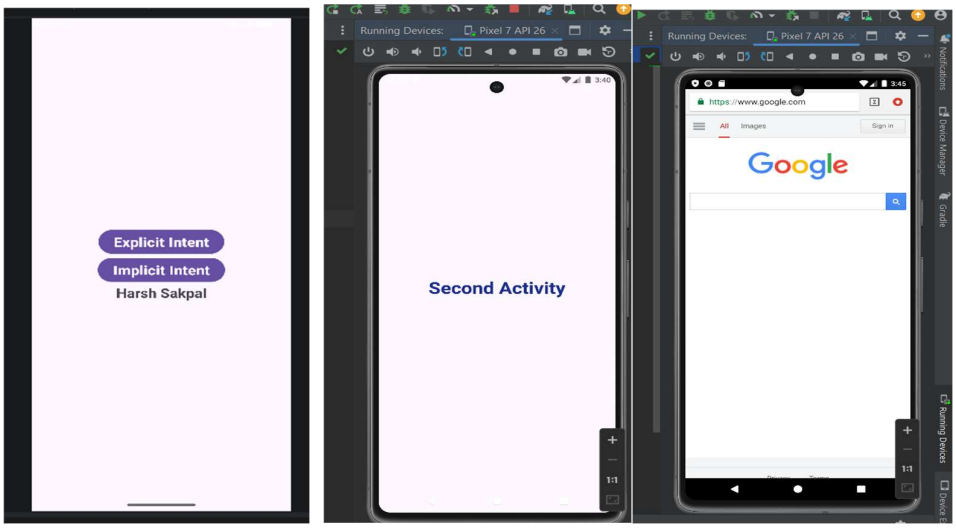
        val btnExplicit = findViewById<Button>(R.id.btnExplicit)
        val btnImplicit = findViewById<Button>(R.id.btnImplicit)

        btnExplicit.setOnClickListener {
            val intent = Intent(this, SecondActivity::class.java)
            startActivity(intent)
        }

        btnImplicit.setOnClickListener {
            val intent = Intent(
                Intent.ACTION_VIEW,
                Uri.parse("https://www.google.com")
            )
            startActivity(intent)
        }
    }
}
```

Output:

Explicit **Implicit**



Practical 5

Aim: Develop an Android application demonstrating Event Handling using Listeners (OnClick, OnLongClick, etc.).

Event handling allows an application to respond to user actions such as clicks, long presses, and selections. Listener interfaces, including OnClickListener and OnLongClickListener, are attached to views to detect these actions. The corresponding callback contains the code that should run when an event occurs.

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <Button
        android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Click Me"
        android:textSize="24sp"
        android:textStyle="bold" />

</LinearLayout>
```

MainActivity.kt

```
package com.example.m2pract5

import android.os.Bundle
import android.widget.Button
import android.widget.Toast
```

```
import androidx.appcompat.app.AppCompatActivity

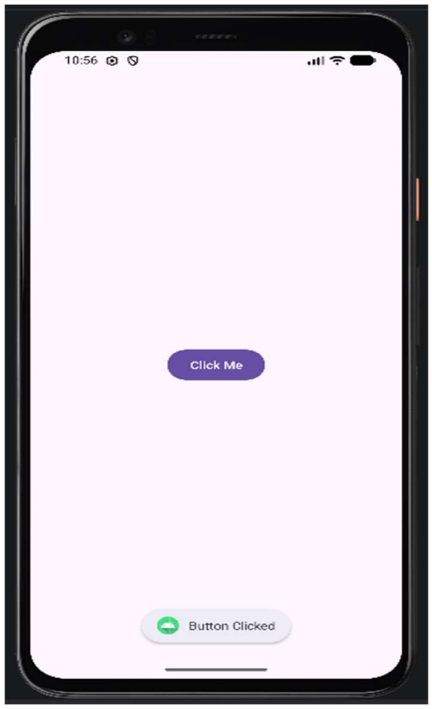
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)sas
        setContentView(R.layout.activity_main)

        val button = findViewById<Button>(R.id.button)

        button.setOnClickListener {
            Toast.makeText(this, "Button Clicked", Toast.LENGTH_SHORT).show()
        }

        button.setOnLongClickListener {
            Toast.makeText(this, "Long Click", Toast.LENGTH_SHORT).show()
            true
        }
    }
}
```

Output:



Practical 8

Aim: Develop an Android application implementing Notifications using Notification Manager.

A notification displays information or alerts to the user outside the application's main screen. NotificationManager is used to issue and manage notifications, while a notification channel is required for most notifications on Android 8.0 and later. The application may need notification permission on newer Android versions.

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Harsh Sakpal"
        android:textColor="#03A9F4"
        android:textSize="24sp"
        android:textStyle="bold|italic" />

    <Button
        android:id="@+id/btnNotify"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Show Notification"
        android:textSize="24sp"
        android:textStyle="bold" />

</LinearLayout>
```

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.M2Pract8"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

MainActivity.kt

```
package com.example.m2pract8

import android.Manifest
import android.app.NotificationChannel
import android.app.NotificationManager
import android.os.Build
import android.os.Bundle
import android.widget.Button
import androidx.appcompat.app.AppCompatActivity
import androidx.core.app.ActivityCompat
import androidx.core.app.NotificationCompat
import androidx.core.app.NotificationManagerCompat

class MainActivity : AppCompatActivity() {

    private val channelId = "my_channel"
```

```
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        createChannel()

        if (Build.VERSION.SDK_INT >= 33) {
            ActivityCompat.requestPermissions(
                this,
                arrayOf(Manifest.permission.POST_NOTIFICATIONS),
                1
            )
        }

        findViewById<Button>(R.id.btnNotify).setOnClickListener {
            showNotification()
        }
    }

    private fun createChannel() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val channel = NotificationChannel(
                channelId,
                "My Notifications",
                NotificationManager.IMPORTANCE_DEFAULT
            )

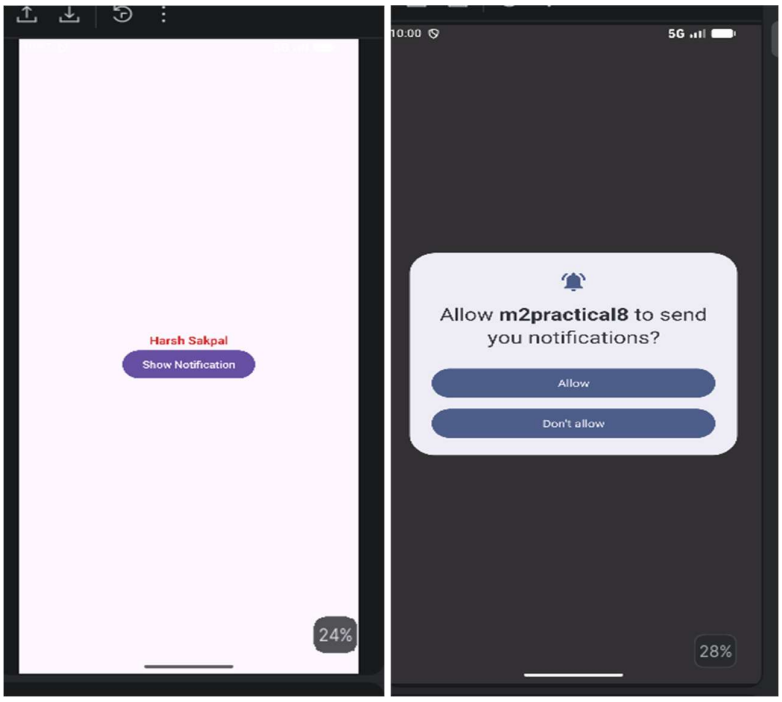
            val manager =
                getSystemService(NotificationManager::class.java)

            manager.createNotificationChannel(channel)
        }
    }

    private fun showNotification() {
        val notification = NotificationCompat.Builder(this, channelId)
            .setSmallIcon(android.R.drawable.ic_dialog_info)
            .setContentTitle("My Notification")
            .setContentText("Hello! This is an Android notification.")
            .setPriority(NotificationCompat.PRIORITY_DEFAULT)
            .setAutoCancel(true)
            .build()

        NotificationManagerCompat.from(this).notify(1, notification)
    }
}
```

Output:


Harsh Sakpal
Show Notification
24%
10:00
5G
Allow m2practical8 to send you notifications?
Allow
Don't allow
28%